CMC Lab 02

Distributed Consistency and Consensus in the Cloud

Lab Documentation Report

Name: Shahd Tamer Abdelmonem (SE3)

---

1. Overview

This lab explores how distributed systems handle consistency, availability, and consensus under different network conditions. Open-source tools Redis and etcd were used to demonstrate replication and the Raft consensus protocol respectively.

2. Objectives

- Simulate eventual consistency with Redis replicas
- Create a network partition and observe replication behavior
- Experiment with Raft consensus using etcd
- Observe leader election and failover in a Raft cluster

3. Environment Setup

3.1 Prerequisites

- Docker Desktop (installed and running)
- Docker Compose
- Windows PowerShell

3.2 Docker Compose Configuration

The following docker-compose.yml was used to spin up the lab environment:

```yaml
services:
  redis-node1:
    image: redis:latest
    container_name: redis-node1
    ports:
      - "6379:6379"

  redis-node2:
    image: redis:latest
    container_name: redis-node2
    command: redis-server --replicaof redis-node1 6379
    ports:
      - "6380:6379"

  etcd:
    image: gcr.io/etcd-development/etcd:v3.5.12
    container_name: etcd
    command:
      - /usr/local/bin/etcd
      - --listen-client-urls=http://0.0.0.0:2379
      - --advertise-client-urls=http://0.0.0.0:2379
    ports:
      - "2379:2379"
```

3.3 Starting the Environment

Navigate to the lab directory and run:

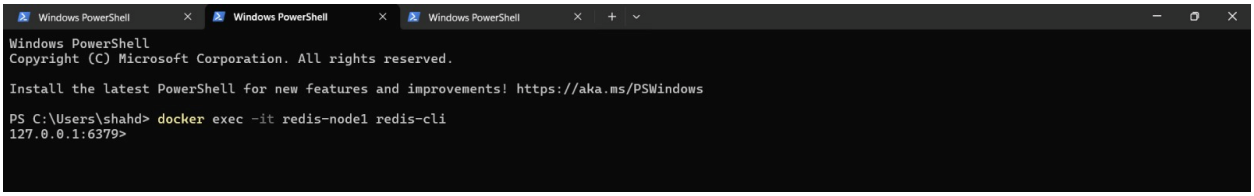
```
cd C:\Users\shahd\OneDrive\Desktop\lab
docker-compose up
```

4. Task 1: Redis Replication and CAP Theorem

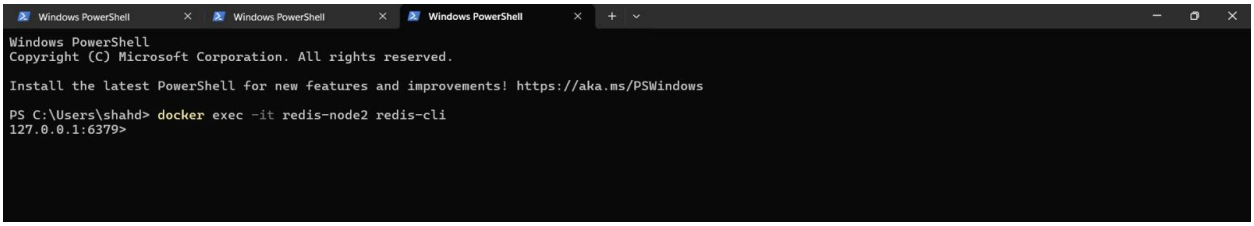
4.1 Connecting to Redis Nodes

Two separate PowerShell windows were opened and connected to each Redis node:

1



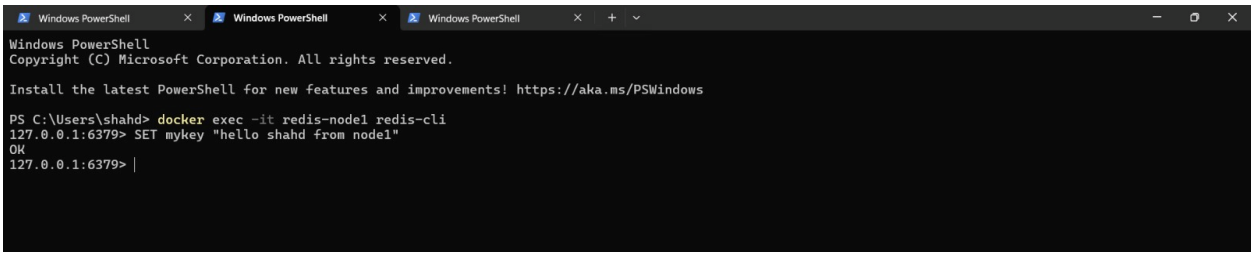
2



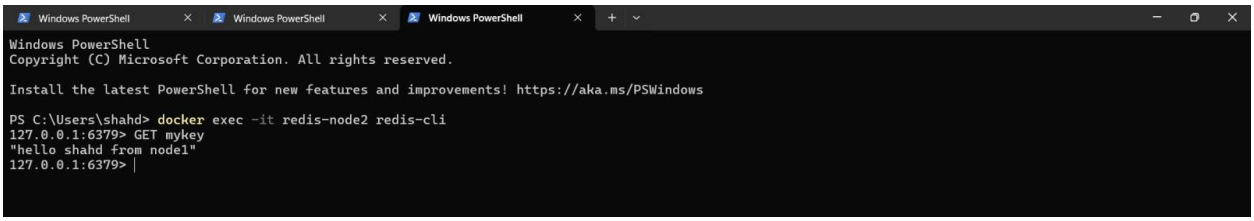
4.2 Writing and Reading Across Nodes

A key was written to redis-node1 and immediately read from redis-node2:

1



2

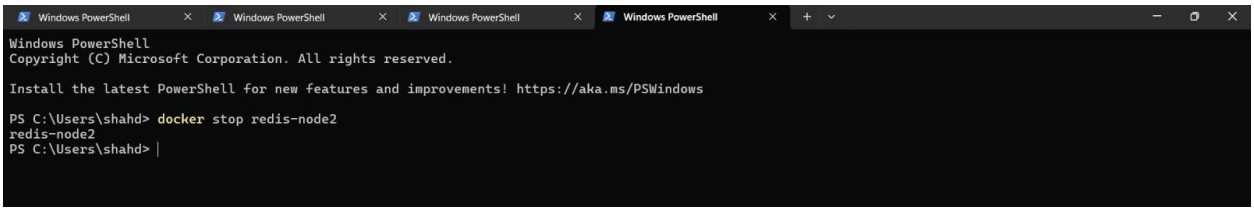


Output	redis-node2 returned "hello shahd from node1" after a brief replication delay.

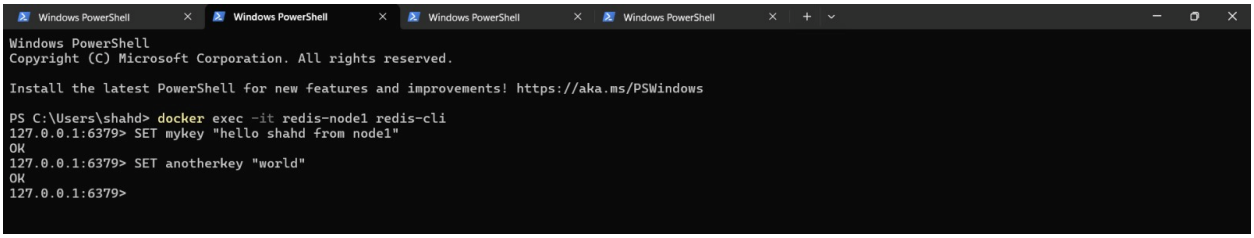
What to observe	Eventual consistency — the replica does not update instantly but catches up shortly after the write.

4.3 Simulating a Network Partition

redis-node2 was stopped to simulate a network failure:



A new key was then written to redis-node1 while node2 was down:

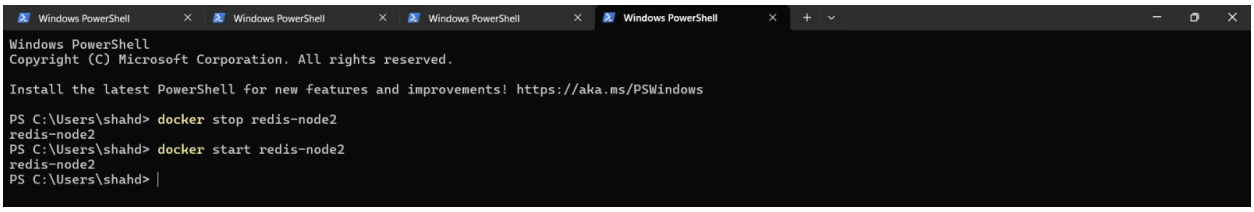


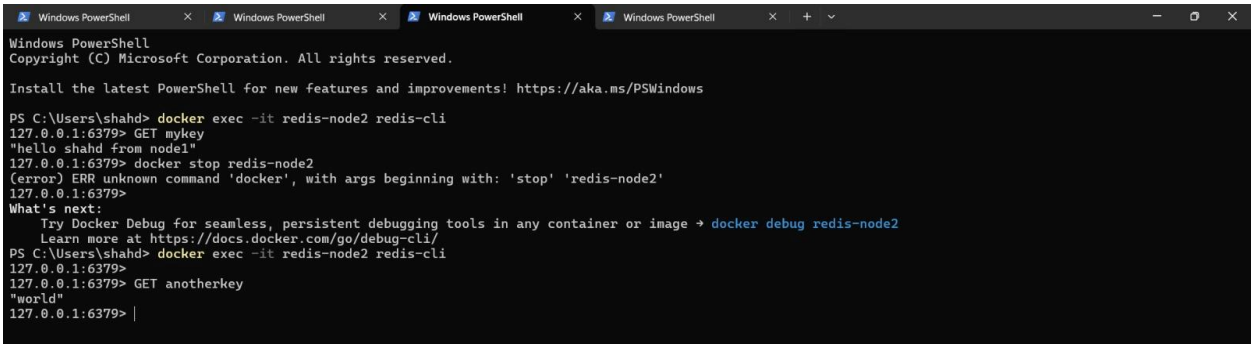
Output	Write to redis-node1 succeeded immediately with OK.

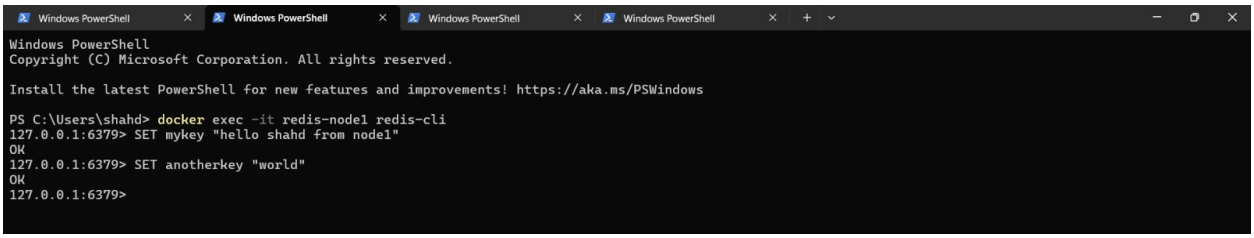
What to observe	Writes to node1 still succeed — Redis prioritizes availability (the "A" in CAP). Node2 is down, so those writes won't replicate until it comes back.

4.4 Recovery After Partition

redis-node2 was restarted and reconnected:







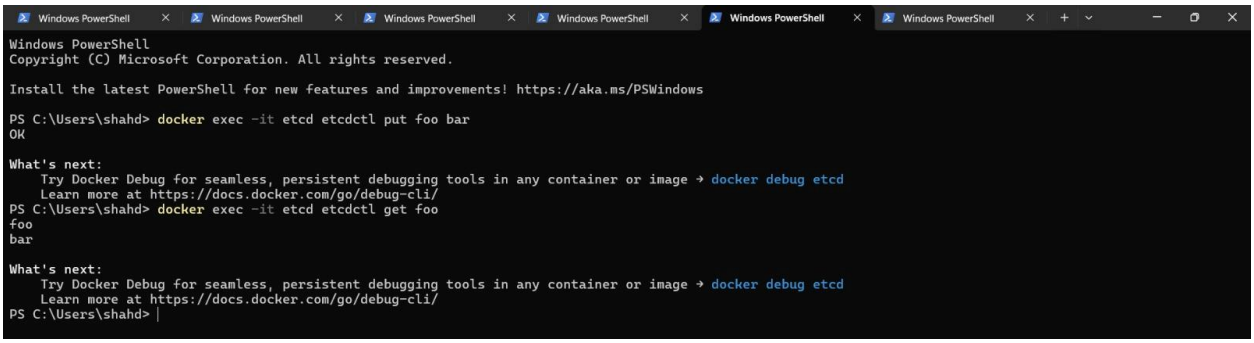
Output	"world" — node2 successfully synced all missed writes from node1.

What to observe	Node2 now syncs and returns "world". This confirms eventual consistency — the data is consistent eventually after the partition heals.

5. Task 2: Raft Consensus with etcd

5.1 Writing and Reading a Key

A key-value pair was written to etcd and read back:

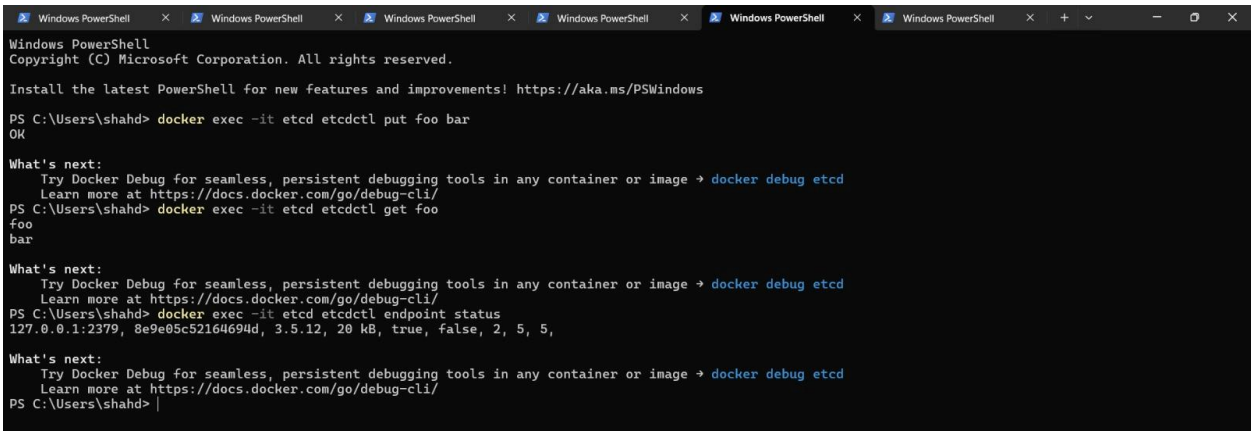


Output	foo bar — the key was confirmed and returned successfully.

What to observe	You get bar back immediately. Unlike Redis replication, etcd uses Raft to guarantee that a write is committed to a majority of nodes before confirming success — this is strong consistency.

5.2 Checking Cluster Leader Status

The endpoint status command was used to inspect the Raft cluster:

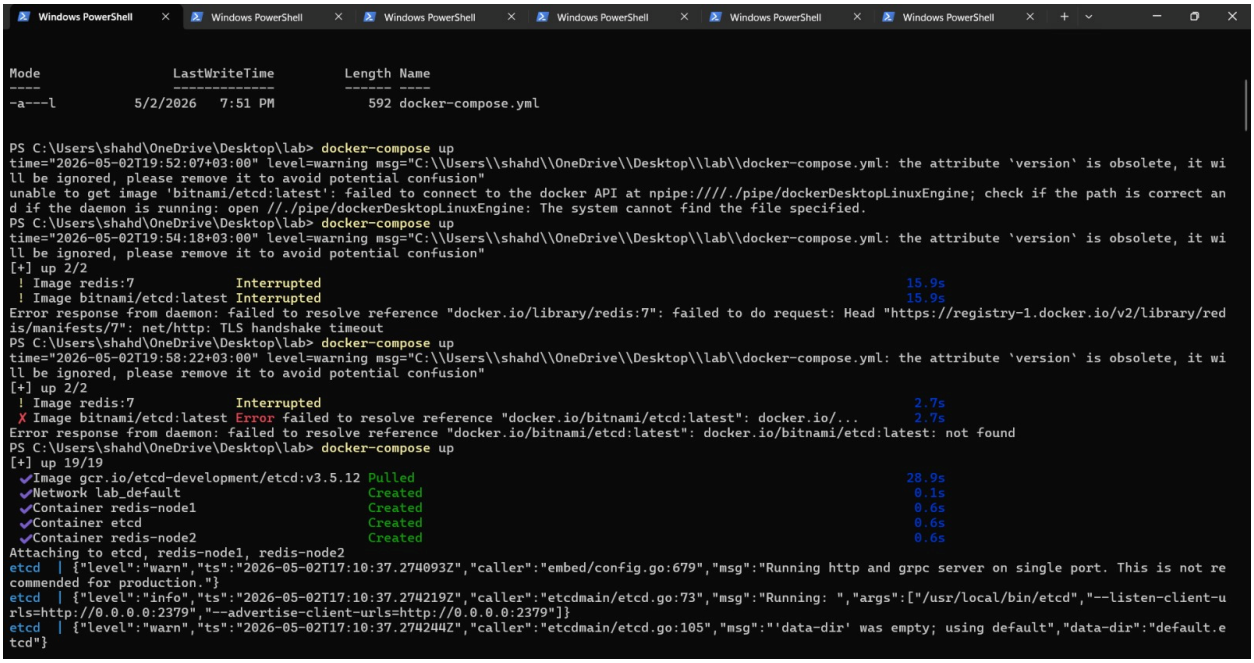


Output	Displays the endpoint URL, node ID, current Raft term, Raft index, and whether the node is the leader.

What to observe	A new leader is elected automatically within a few seconds. This is Raft's leader election — it ensures the cluster keeps working as long as a majority (quorum) of nodes are alive.

5.3 Simulating Leader Failure

The etcd container was stopped to simulate a leader crash:



Output	After restarting, etcd rejoined the cluster. The Raft term incremented, indicating a new election took place.

What to observe	Raft guarantees automatic leader re-election when the current leader becomes unavailable, ensuring the cluster remains operational as long as a majority of nodes are alive.

6. Summary of Findings

Concept	Tool Used	Observed Behavior
Eventual Consistency	Redis	Node2 briefly returned nil before syncing with node1
Availability (CAP)	Redis	Node1 continued accepting writes while node2 was down
Data Recovery	Redis	Node2 re-synced all missed writes upon restart
Strong Consistency	etcd	Write confirmed only after Raft majority agreement
Leader Election	etcd	New leader elected automatically after container stop

7. Conclusion

This lab demonstrated the practical trade-offs described by the CAP theorem through hands-on experimentation with Redis and etcd.

Redis, configured in a primary-replica setup, prioritizes Availability — it continues to function and accept writes even when replicas are unreachable, eventually synchronizing data once connectivity is restored. This makes Redis well-suited for use cases where uptime is critical and brief data inconsistency is tolerable.

etcd, using the Raft consensus protocol, prioritizes Consistency — every write is confirmed only after agreement from a majority of cluster nodes. This guarantees that all reads reflect the most recent committed state, making etcd ideal for critical coordination tasks such as service discovery, distributed locking, and configuration management.

Understanding these trade-offs is essential when designing distributed systems, as the choice of consistency model directly impacts the system's behavior during network failures and node outages.

8. Cleanup

After completing all tasks, the lab environment was shut down: